

Week 5 – File Permissions and Linux Processes

Week 4 is not included in the uploaded summaries, so the next available topic is **Week 5**.

Topics covered

- RH124 Chapter 7: Controlling Access to Files
 - RH124 Chapter 8: Monitoring and Managing Linux Processes
 - File ownership
 - Read, write and execute permissions
 - Symbolic and numerical **chmod**
 - Special permissions
 - Default permissions using **umask**
 - Viewing and controlling processes
 - Foreground and background jobs
 - Sending signals with **kill**
-

1. Viewing file ownership and permissions

Use:

```
ls -l filename
```

For a directory itself:

```
ls -ld directory
```

Example output:

```
-rw-r--r-- 1 student student 120 Jul 23 10:30 notes.txt
```

The important fields are:

```
1. - rw-r--r-- student student
2. | |         |         |
3. | |         |         | Group owner
4. | |         |         | User owner
5. | |         |         | Permissions
   | |         |         |
   | |         |         | File type
```

The first character indicates the file type:

Character	Meaning
-----------	---------

-	Regular file
---	--------------

d	Directory
---	-----------

l	Symbolic link
---	---------------

2. File ownership

Every file has:

- A **user owner**
- A **group owner**

The user who creates a file normally becomes its owner. The group owner is normally the user's primary group.

Check your user and group details:

id

Example output:

```
uid=1001(student) gid=1001(student) groups=1001(student),10(wheel)
```

This shows:

- UID: 1001
 - Primary group: student
 - Supplementary group: wheel
-

3. Changing the user owner

Use **chown**:

```
sudo chown username filename
```

Example:

6. touch testfile
7. ls -l testfile
8. sudo chown root testfile

```
ls -l testfile
```

Expected change:

9. Before: student student

After: root student

Only root can normally change a file's user owner to another user.

4. Changing the group owner

Use:

```
chgrp groupname filename
```

Example:

```
sudo chgrp wheel testfile
```

Verify:

```
ls -l testfile
```

You can also change both user and group owners with **chown**:

```
sudo chown student:wheel testfile
```

Recursive ownership change:

```
sudo chown -R student:wheel project/
```

The **-R** option applies the change to the directory and everything inside it.

5. Understanding **r**, **w** and **x**

Permissions are divided into three groups:

10. **rw** **rw** **rw**
11. | | |
12. | | └─ Others
13. | └─── Group owner
 └─── User owner

The three permission types are:

Permission	File meaning	Directory meaning
r	Read or copy the file	List filenames inside
w	Modify the file	Create or delete entries
x	Execute the file	Enter and access the directory

For directories, write permission normally requires execute permission to function properly.

Example:

-rwxr-x---

This means:

Cate g or y	Permiss ions
----------------------	-----------------

Owne r	<code>rwX</code>
-----------	------------------

Grou p	<code>r-X</code>
-----------	------------------

Other s	<code>---</code>
------------	------------------

6. Permission-checking order

Linux checks permissions in this order:

1. Are you the file owner?
2. If not, are you a member of the group owner?
3. If not, use the permissions for others.

Linux does not combine all three sets.

For example:

```
-r--rw-rwx student developers file.txt
```

If you are the owner `student`, you only receive `r--`, even though others have `rwX`.

7. Changing permissions symbolically

General format:

```
chmod category operation permission filename
```

Categories:

Sy	Meaning
m	
b	
o	
l	

u	User owne r
---	-------------------

g	Group owne r
---	--------------------

o	Others
---	--------

a	All
---	-----

Operations:

Sy	Meaning
m	
b	
o	
l	

+	Add permission
---	----------------

- Remove
permission

= Set exact
permission

Permissions:

Sy	Mea
m	n
b	i
o	n
l	g

r Read

w Write

x Exec
ut
e

Add execute permission for the owner:

```
chmod u+x script.sh
```

Remove write permission from the group:

```
chmod g-w file.txt
```

Give others read permission:

```
chmod o+r file.txt
```

Set exact owner permissions:

```
chmod u=rw file.txt
```

Make several changes at once:

```
chmod u+x,g-w,o+r file.txt
```

Verify after every change:

```
ls -l file.txt
```

8. Recursive permissions

Apply permissions to a directory and everything inside it:

```
chmod -R g+r project/
```

Be careful with:

```
chmod -R a+x project/
```

This adds execute permission to ordinary files as well.

The slides recommend using capital **X** when you only want execute permission on directories and files that are already executable:

```
chmod -R g+X project/
```

This gives group execute permission to directories without unnecessarily making ordinary files executable.

9. Numerical permissions

Numerical permission values are:

Permis sio n	V
--------------------	---

Read 4

Write 2

Execute 1

Add the values together:

**V Permiss
 ions**

0 ---

1 --x

2 -w-

3 -wx

4 r--

5 r-x

6 rw-

7 rwx

Example:

```
chmod 754 file.txt
```

This gives:

Cate g or y	V	Permiss ions
----------------------	---	-----------------

Owne r	7	rwx
-----------	---	-----

Grou p	5	r-x
-----------	---	-----

Other s	4	r--
------------	---	-----

Result:

```
-rwxr-xr--
```

Common permission settings:

Permis sio n	Meaning
600	Owner can read and write only
640	Owner read/write, group read
644	Normal non-executable file
700	Private executable or directory
750	Owner full, group read/execute
755	Common directory or executable
770	Owner and group full access
777	Everyone has full access

Avoid 777 unless it is specifically required.

10. Permission scenarios

Owner needs full access, group needs read access, others no access

```
chmod 740 file.txt
```

Result:

```
-rwxr-----
```

Owner and group need read/write, others need read only

```
chmod 664 file.txt
```

Result:

```
-rw-rw-r--
```

Private directory for one user

```
chmod 700 private/
```

Result:

```
drwx-----
```

Shared directory for owner and group

```
chmod 770 shared/
```

Result:

```
drwxrwx---
```

11. Special permissions

Linux supports three special permissions:

Permis sio n	Numerical value	Typical use
Setuid	4000	Execute using file owner privileges

Setgid	2000	Execute using group privileges or inherit directory group
Sticky bit	1000	Restrict deletion in shared directories

12. Setuid

Setuid is applied to executable files. When executed, the program runs using the file owner's privileges.

Symbolic:

```
chmod u+s program
```

Numerical:

```
chmod 4755 program
```

Example output:

```
-rwsr-xr-x
```

The **s** appears in the owner execute position.

A common example is the **passwd** command, which requires controlled access to protected password-related files.

Find setuid files:

```
find / -perm -4000 2>/dev/null
```

Setuid should be used carefully because it may create privilege-escalation risks.

13. Setgid

On an executable file, setgid causes it to run using the group owner's privileges.

On a directory, newly created files inherit the directory's group owner.

Set it symbolically:

```
chmod g+s shared/
```

Numerically:

```
chmod 2770 shared/
```

Example output:

```
drwxrws---
```

This is useful for shared project directories because all files created inside inherit the same group.

14. Sticky bit

The sticky bit is mainly used on shared directories.

Without it, users with write and execute permission on a directory may delete files belonging to other users.

With the sticky bit, a file can normally be deleted only by:

- The file owner
- The directory owner
- Root

Symbolic:

```
chmod o+t shared/
```

Numerical:

```
chmod 1777 shared/
```

Example:

```
drwxrwxrwt
```

The **t** appears in the others execute position.

Common examples include:

```
14. /tmp
```

/var/tmp

15. Default permissions and **umask**

Linux calculates default permissions using **umask**.

View the current value:

```
umask
```

Common output:

```
0022
```

Maximum default permissions are:

Object	Maximum default
File	666
Directory	777

Files do not receive execute permission by default.

With a **umask** of 022:

15. File: $666 - 022 = 644$

Directory: $777 - 022 = 755$

Results:

16. File: `rw-r--r--`

Directory: `rw-r-xr-x`

Set a temporary **umask**:

```
umask 027
```

Expected defaults:

17. File: 640

Directory: 750

Test it:

18. `umask 027`

19. `touch testfile`

20. `mkdir testdir`

`ls -ld testfile testdir`

Expected output:

21. `-rw-r----- testfile`

`drwxr-x--- testdir`

Set a highly restrictive mask:

```
umask 077
```

Expected defaults:

22. File: 600

Directory: 700

To make a **umask** persistent, it may be added to:

```
~/.bashrc
```

16. Understanding processes

A process is a running instance of a command or program.

Every process has information such as:

- PID: Process ID

- PPID: Parent Process ID
- User account
- CPU use
- Memory use
- Current state
- Command

Processes run using the privileges of the account that started them.

17. Viewing processes with **ps**

Show processes attached to the current terminal:

```
ps
```

Detailed process list:

```
ps -ef
```

BSD-style detailed list:

```
ps aux
```

Search for a particular process:

```
ps aux | grep ssh
```

Useful fields include:

Field	Meaning
-------	---------

USER	Process owner
------	---------------

PID	Process ID
-----	------------

%CPU	CPU usage
%MEM	Memory usage
STAT	Process state
COMMAND	Command and arguments

18. Process states

Common process-state letters include:

S	Meaning
---	---------

R	Running
---	---------

S	Interruptible sleep
---	---------------------

D Uninterruptible
 sleep

T Stopped

Z Zombie

X Dead

A sleeping process is not necessarily faulty. Many processes wait until an event occurs.

A zombie process has completed but still has an entry in the process table because its parent has not collected its exit status.

19. Monitoring processes with **top**

Run:

`top`

top continuously displays:

- System uptime
- Number of tasks
- CPU use
- Memory use
- Swap use
- Process list

Useful keys inside **top**:

K **Action**

P Sort by CPU usage

M Sort by memory
usage

k Send a signal to a
process

r Change process
priority

q Quit

A single non-interactive snapshot can be produced with:

```
top -b -n 1
```

20. Foreground and background processes

A foreground process occupies the terminal until it finishes.

Example:

```
sleep 300
```

The prompt does not return until the command finishes or is stopped.

Start a process directly in the background:

```
sleep 300 &
```

Expected output:

```
[1] 303
```

Here:

- 1 is the job number
 - 303 is the PID
-

21. Viewing jobs

Use:

```
jobs
```

This displays jobs started from the current shell.

Example:

```
[1]+  Running sleep 300 &
```

The `jobs` command only displays jobs belonging to the current terminal session.

Display job PIDs:

```
jobs -l
```

22. Suspending a foreground process

Start:

```
sleep 300
```

Press:

```
Ctrl+Z
```

The process becomes stopped or suspended.

Check:

```
jobs
```

Example:

```
[1]+  Stopped sleep 300
```

23. Moving jobs between foreground and background

Continue the most recent stopped job in the background:

```
bg
```

Or specify the job:

```
bg %1
```

Bring it back to the foreground:

```
fg %1
```

Job numbers use %, while PIDs do not.

Example:

```
fg %2
```

refers to job 2.

24. Ending foreground commands

Press:

```
Ctrl+C
```

This normally sends the **SIGINT** signal and requests that the process terminate.

Ctrl+Z does not terminate a process. It suspends it.

25. Sending signals with **kill**

General syntax:

```
kill PID
```

By default, **kill** sends signal 15, **SIGTERM**.

Example:

```
23. sleep 500 &
```

24. jobs -l

kill 1234

Replace 1234 with the actual PID.

Explicitly send SIGTERM:

kill -15 1234

or:

kill -TERM 1234

Verify:

ps -p 1234

26. Common signals

Sig	Num	Meaning
n	b	
a	e	
l	r	
SIG	1	Reload or hang
H		up
U		
P		
SIG	2	Interrupt
I		
N		
T		
SIG	3	Quit
Q		

U
I
T

SIG	9	Force
K		termination
I		
L		
L		

SIG	15	Normal
T		termination
E		
R		
M		

SIG	19	Stop process
S		
T		
O		
P		

SIG	18	Continue
C		process
O		
N		
T		

List all signals:

kill -l

27. SIGTERM versus SIGKILL

Use this first:

```
kill PID
```

or:

```
kill -15 PID
```

This gives the process an opportunity to close files and clean up.

Use this only when the process refuses to terminate:

```
kill -9 PID
```

SIGKILL immediately forces the process to stop and cannot be ignored. The slides recommend using it only as a last resort.

28. Ending processes by name

Terminate processes using a command name:

```
pkill sleep
```

Terminate all processes with the exact name:

```
killall sleep
```

These commands may affect multiple matching processes, so verify first:

```
pgrep -a sleep
```

29. Process priorities

Linux process priority uses a **nice value**.

Typical nice-value range:

-20 to 19

Nice value	Meaning
-20	Highest priority
0	Normal priority
19	Lowest priority

A higher nice number means the process behaves more “nicely” by receiving lower CPU priority.

Start a process with a nice value:

```
nice -n 10 command
```

Example:

```
nice -n 10 sleep 500 &
```

View it:

```
ps -o pid,ni,cmd -C sleep
```

30. Changing an existing process priority

Use **renice**:

```
renice 15 -p PID
```

Example:

```
25. sleep 500 &
```

```
26. jobs -l
```

```
renice 15 -p 1234
```

A normal user can normally lower the priority of their own processes by increasing the nice value.

Administrative privileges are required to increase priority by assigning a lower or negative nice value:

```
sudo renice -5 -p 1234
```

27.